

Arrays em Julia e Pluto

Depois de organizar coleções de dados em teoria (Parte 1 — Conceitos), chegou a hora de praticar cada ideia como código executável, no Pluto.



O que é e por que usar um Array?

Conceito

Uma estrutura que armazena vários dados do mesmo tipo em uma única variável.

Por que usar?

- Organiza informações de forma eficiente, evitando criar muitas variáveis individuais.
- Facilita o acesso, a busca e a modificação de dados.

Estrutura

- Índices: posições numeradas que identificam cada elemento.
- Elementos: os valores armazenados em cada posição.



Exemplo cotidiano

Uma prateleira com várias caixas numeradas, cada uma guardando um item.

EXEMPLO DE CÓDIGO

```
nomes = ["Ana", "João", "Maria", "Lucas"]
```

```
nomes[0] → "Ana"
```

```
nomes[2] → "Maria"
```



Dimensões de um Array



1D — Unidimensional

Armazena uma sequência linear de elementos.

Exemplo: Lista com os nomes dos alunos de uma turma.

```
alunos = ["Ana", "João",  
          "Maria", "Lucas"]  
print(alunos[2])  
# Exibe: Maria
```

Uso cotidiano

Lista de contatos do celular ou lista de músicas em um app.



2D — Bidimensional

Armazena dados em linhas e colunas, como uma planilha.

Exemplo: Notas dos alunos em diferentes provas.

```
notas = [[8.5, 9.0, 7.5],  
         [7.0, 8.0, 6.5],  
         [9.5, 8.5, 10.0]]  
print(notas[1][2])  
# Exibe: 6.5
```

Uso cotidiano

Tabela de horários de ônibus ou boletim escolar.



Multidimensional

Armazena dados em mais de duas dimensões.

Exemplo: Representação de uma imagem (cada pixel tem R, G, B).

```
imagem = [[[255, 0, 0],  
           [0, 255, 0]], [[0, 0, 255],  
                          [255, 255, 0]]]  
print(imagem[0][1])  
# Exibe: [0, 255, 0]
```

Uso cotidiano

Fotos digitais, mapas, planilhas 3D ou jogos (x, y, z).



Aninhamento: Arrays de Arrays

Um array aninhado é um conjunto dentro de outro conjunto — uma lista que contém outras listas. É uma forma de organizar informações em camadas, como caixas dentro de caixas.



Notas dos alunos

```
Ana    → [8.5, 9.0, 7.5]
João   → [7.0, 6.5, 8.0]
Maria  → [10.0, 9.5, 9.0]
```

O conjunto de listas forma um array bidimensional (array de arrays) que representa uma turma inteira.



Cardápio do restaurante

```
Pratos → [Lasanha, Feijoadá, Peixe]
Sobremesas → [Pudim, Sorvete, Bolo]
Bebidas → [Suco, Refri, Café]
```

O cardápio completo é um array de arrays: guarda várias listas dentro de uma só.



Mala de viagem

```
Saco 1 → [camisas, calças]
Saco 2 → [meias, sapatos]
Saco 3 → [produtos de higiene]
```

A mala é um array e cada saco é outro array: juntos formam um array aninhado.



Arrays como Contêineres Universais



Flexibilidade total

Um array pode ser visto como um contêiner universal porque guarda diferentes elementos organizados em sequência. Possui estrutura consistente — funciona do mesmo jeito independentemente do conteúdo (números, textos, imagens) — mantendo o mesmo formato, índice e operações básicas: acessar, adicionar, remover, percorrer. Essa consistência torna os arrays previsíveis e fáceis de usar, permitindo aplicar os mesmos algoritmos a diferentes tipos de dados.



Poder computacional

Os arrays concentram grande poder computacional porque permitem armazenar e processar grandes quantidades de dados de forma organizada e eficiente. Sua estrutura uniforme possibilita operações rápidas e repetitivas — busca, ordenação, análise em massa — essencial para o pensamento computacional, pois favorece algoritmos lógicos, escaláveis e reutilizáveis.

O que já vimos na Parte 1

- Arrays organizam coleções de dados do mesmo tipo em índices numerados — de listas simples (1D) a estruturas em linhas e colunas (2D) e além (multidimensionais).
- Arrays podem se aninhar — arrays dentro de arrays — para representar informação em camadas, como turmas, cardápios ou malas organizadas.
- Por sua estrutura consistente, arrays são contêineres universais: previsíveis, reutilizáveis e essenciais para algoritmos escaláveis.

Agora, a mesma ideia — só que executável: cada conceito da Parte 1 vira uma tarefa curta no Pluto, com um exemplo real em Julia.

Como um array guarda uma sequência de itens?

NO PLUTO

- Crie um array com os nomes dos alunos de uma turma.
- Acesse o terceiro nome.
- Percorra o array com um laço, imprimindo cada nome.

Diferença da Parte 1: em Julia, os índices começam em 1, não em 0 — `alunos[3]` é o terceiro nome, não o quarto.

```
alunos = ["Ana", "João", "Maria", "Lucas"]  
terceiro = alunos[3]  
maiusculos = [uppercase(nome)  
               for nome in alunos]
```

```
alunos[3] → "Maria"  
maiusculos → ["ANA", "JOÃO", "MARIA", "LUCAS"]
```

Como representar dados em linhas e colunas?

NO PLUTO

- Represente as notas dos alunos em 3 provas como uma matriz.
- Acesse a nota da Maria (linha 3) na prova 3 (coluna 3).

```
notas = [8.5 9.0 7.5;  
        7.0 8.0 6.5;  
        9.5 8.5 10.0]  
nota_maria_prova3 = notas[3, 3]
```

Diferença da Parte 1: Julia usa uma matriz nativa — `notas[linha, coluna]`, com vírgula — em vez de colchetes encadeados como `notas[1][2]`.

```
notas[3, 3] → 10.0
```

Como somar todos os valores de uma matriz?

NO PLUTO

- Some todas as notas da matriz (mesmos dados da Tarefa 13.2).
- Use dois laços for aninhados: um para linha, um para coluna.
- Compare o resultado com a função `sum` de Julia.

```
total_manual = 0.0
for linha in 1:size(notas, 1)
    for coluna in 1:size(notas, 2)
        total_manual += notas[linha, coluna]
    end
end
```

```
total_manual → 74.5
sum(notas) → 74.5 (mesmo valor)
```

Como um array representa uma imagem?

NO PLUTO

- Represente uma imagem 2×2 onde cada pixel tem 3 valores de cor (R, G, B).
- Acesse a cor do pixel da linha 1, coluna 2.

Aqui a imagem é um array de arrays de arrays (3 níveis) — a mesma ideia de aninhamento da Parte 1, um nível a mais.

```
imagem = [  
    [[255, 0, 0], [0, 255, 0]],  
    [[0, 0, 255], [255, 255, 0]],  
]  
pixel_1_2 = imagem[1][2]
```

`pixel_1_2` → [0, 255, 0] (verde)

SÍNTESE DA AULA

Arrays em Julia, na prática

- Em Julia, os arrays começam no índice 1, e matrizes usam vírgula (linha, coluna) em vez de colchetes encadeados.
- O mesmo percurso conceitual da Parte 1 — 1D, 2D e aninhado/3D — se traduz diretamente em código executável no Pluto.
- Percorrer, somar e comparar com funções prontas (como `sum`) ajuda a verificar se o raciocínio manual está certo.

A SEGUIR

Aula 14: Complexidade de Algoritmos

REFERÊNCIAS

Referências

- RIBEIRO, João Araújo. Introdução à Programação e aos algoritmos. Rio de Janeiro: LTC, 2019. ISBN 978-85-216-3640-3.